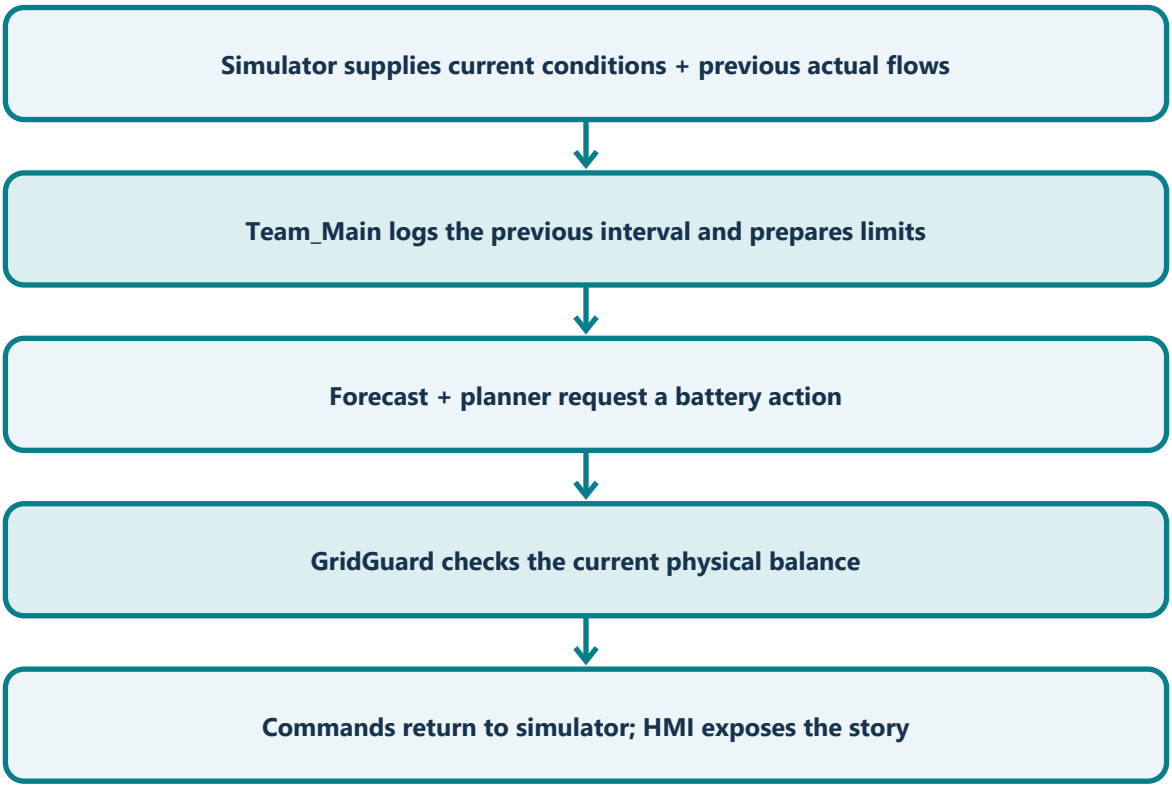
PLC HACKATHON • VERIFIED V7

Understand your energy manager

A practical guide to the code you wrote: what it observes, how it chooses battery power, how it protects the grid, and how it proves the result.



The version this guide explains

Frozen source: deliver/ME_v7_src/ — the verified v7 cap-floor version. It does not describe later code optimisations or the evolving WebVisu screen project.

Scenario	Verified annual cost	Safety / EV
S1 — two tariff	€204,598.62	No trip / 100% EV
S2 — day ahead	€169,833.86	No trip / 100% EV

Read pages 3–7 first for the essentials. Pages 8–22 explain the controller. The final reference pages connect every source file to its role.

LEARNING GUIDE

A route through the guide

Pages	What you learn
3–5	Power, energy, sign conventions, simulated time, and how to read ST
6–8	Team_Main, feasible physical limits, EV policy and strategy setup
9–10	How the code learns a forecast without embedded load/PV profiles
11–15	Peak cap, dynamic programming, first action, and v7 cap floor
16–17	Grid protection and a numerical worked example
18–20	Energy/cost accounting and the EV-departure workaround
21–23	HMI data, trends/alarms, tests, results and practical limitations
24–25	Every source file, grouped by responsibility
26 onward	Actual source excerpts for supporting files and tuning constants

Three things to keep separate

Simulator: the organiser's model of the plant and its measurements. Controller: our code that chooses setpoints. HMI: the screen that displays values and sends permitted control actions. A browser does not run the optimisation.

Pseudocode in this guide is simplified English-like logic. Boxes labelled Actual ST are copied from the frozen source with original line numbers. Diagrams are vector flowcharts; editable Mermaid versions are saved beside the generator.

LEARNING GUIDE

1 · The plant and the power balance

A site consumes electricity, generates solar power, charges vehicles, and has a battery. The controller chooses battery charging/discharging and sometimes reduces solar output. It tries to reduce the annual bill while delivering EV energy and avoiding a grid trip.

Symbol / acronym	Meaning and units
FNE / PV	Fotonaponska elektrana: solar generation, kW
FEC	The site's fixed consumption/load, kW
BESS	Battery energy storage system: 1500 kWh nominal, ±750 kW
SoC	State of charge, %. Allowed range 10–90%
EV	Electric vehicle; four chargers, up to 22 kW each
EGC / grid	Grid connection: import up to 1000 kW, export up to 499 kW
kW versus kWh	kW is power now; kWh is energy accumulated over time

The key sign convention

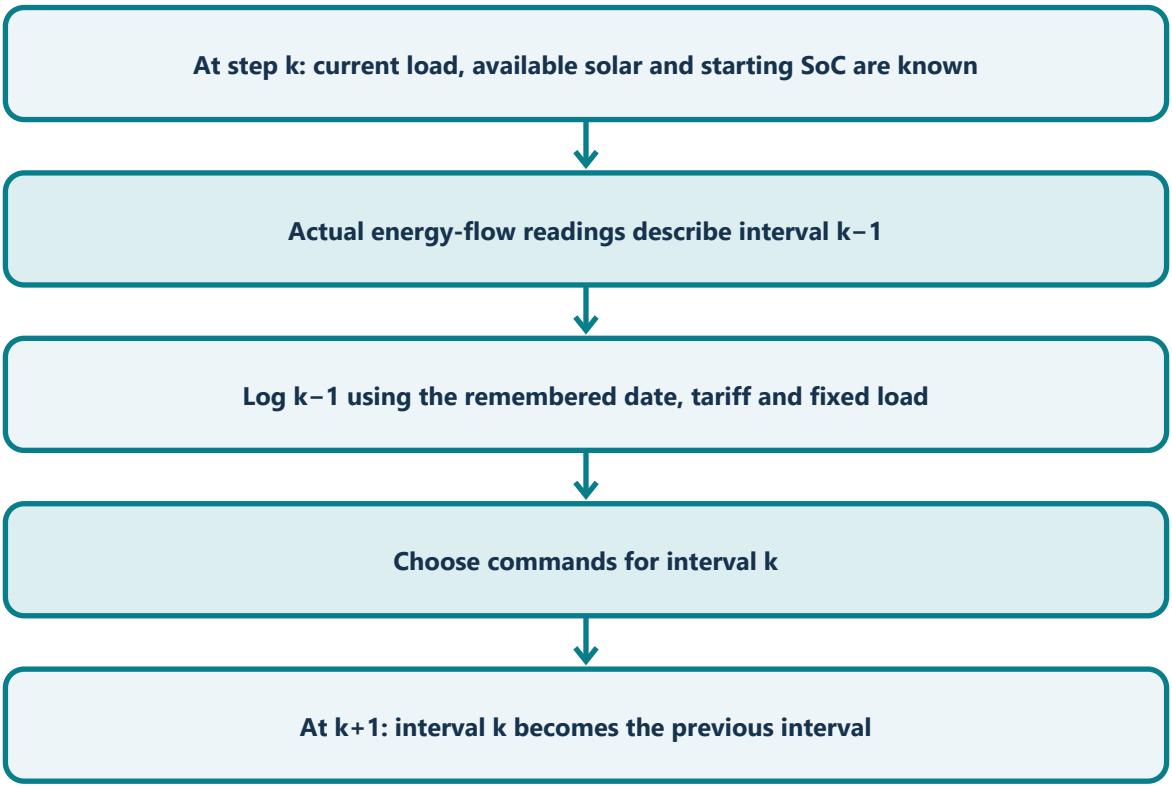
```
grid = fixed_load - solar + sum(EV_power) - battery_power

battery_power > 0  means DISCHARGE into the site
battery_power < 0  means CHARGE from the site
grid > 0           means IMPORT
grid < 0           means EXPORT
```

At 400 kW for one 15-minute interval, energy = 400 × 0.25 = 100 kWh. Battery losses mean that bus energy and stored battery energy are different.

2 · Two clocks, one correct sequence

One simulated interval represents 15 minutes, but the computer may calculate it in milliseconds. The control result must depend on the simulated step k, not the laptop clock or how quickly the PLC task repeats.



There are 96 intervals per day and 35,040 per year. At k = 0, reset the controller and logs. At k = 35,040, log the last interval and issue zero commands. Do not accidentally skip that final bookkeeping call.

A normal simulation period is at least 10 ms. Earlier validation also used a 5 ms stress setting; that is a performance check, not the permitted normal setting.

LEARNING GUIDE

3 · Read Structured Text without fear

ST notation	Plain meaning
FUNCTION_BLOCK	A reusable object with memory between calls: forecasts and logs retain state
FUNCTION	A helper that calculates a return value; see F_PeakCap and F_StageCost
VAR_INPUT / VAR_OUTPUT	Values passed in / values produced by the block
VAR_IN_OUT	An argument passed by reference, used here for large arrays
:= / = / <>	Assign a value / compare equality / compare inequality
IF … THEN … END_IF	Do a block only when a condition is true
FOR / WHILE	Repeat instructions; each repetition costs execution time
REAL / LREAL	Floating-point number / higher precision floating-point number
LIMIT(lo, x, hi)	Clamp x so it stays between lo and hi
SEL(flag, a, b)	Returns b when flag is TRUE, otherwise a

A name is a clue: r often marks a REAL, x a BOOL, a an array, i an integer and fb a function-block instance. Req means requested, Act actual, and Prev previous.

Actual ST • F_DayType.st • lines 7–13

```
007  IF iWeekday = 6 THEN
008   F_DayType := 1;
009  ELSIF iWeekday = 7 THEN
010   F_DayType := 2;
011  ELSE
012   F_DayType := 0;
013  END_IF;
```

This helper groups Monday–Friday together, Saturday separately, and Sunday separately. The forecast keeps different load profiles for those three day types.

LEARNING GUIDE

4 · Team_Main is the conductor

Team_Main in pseudocode

```
if this is a new simulated step:
  if k == 0: reset state and logs
  if previous interval exists and k advanced by exactly one:
    correct EV departure measurements
    log previous energy, then previous cost
  if simulation is finished:
    set every command to zero
  else:
    calculate feasible battery / EV limits
    ask Strategy for requested setpoints
    pass those requests through GridGuard
    write a complete command from the guarded values
    remember inputs needed at k+1
update HMI support data
return the held command
```

Actual ST • Team_Main.st • lines 63–70

```
063  k := simOut.t_sim.k;
064  xNewStep := (NOT xInit) OR (k <> kLast) OR (simOut.t_sim.stamp <> dtLast);
065
066  IF xNewStep THEN
067   xInit := TRUE;
068   xResetStep := (k = 0);
069   IF xResetStep THEN
070     xHavePrev := FALSE;
```

The new-step gate prevents double-counting and repeatedly learning the same observation when the PLC calls the block again. It notices changes in k or the simulated timestamp. The remembered command is still returned on every call.

Notice the order: the energy logger updates monthly peaks before the cost logger prices them; the strategy proposes an action before the guard corrects it. Changing that order changes behaviour.

LEARNING GUIDE

5 · Physical limits and the EV promise

Before optimisation, Team_Main calculates what the simulator can actually accept. Near the battery's upper/lower SoC boundary, its feasible power can be much smaller than 750 kW.

Feasible bus-side limits

```
usable battery energy = (SoC − minimum SoC) / 100 × capacity
max charge = min(750, remaining room / (0.25 × charge efficiency))
max discharge = min(750, usable energy × discharge efficiency / 0.25)

EV feasible power = min(22, remaining energy / (0.25 × 0.9))
```

With 1500 kWh capacity and a 10–90% SoC window, the usable battery window is 1200 kWh. Charge and discharge efficiencies are 0.95 each. EV efficiency is 0.9.

Actual ST • FB_Strategy.st • lines 55–61

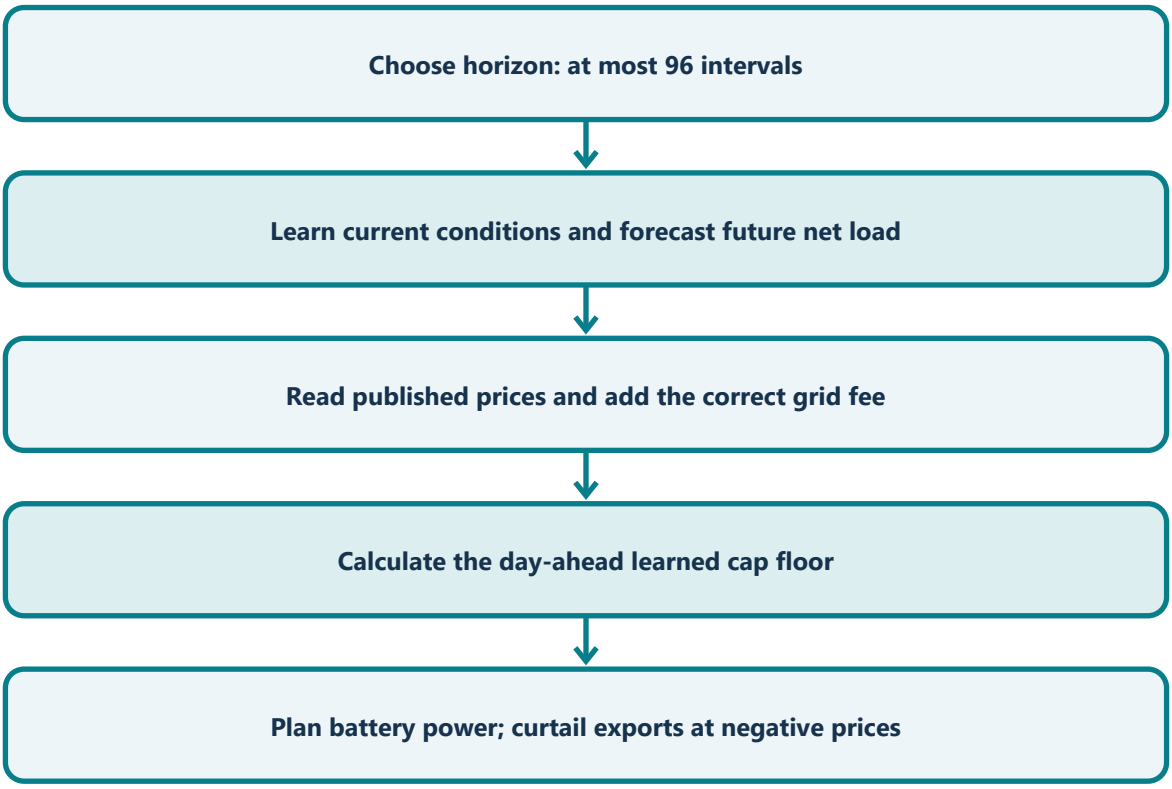
```
055  sumEv := 0.0;
056  FOR j := 0 TO 3 DO
057   aEvReq[j] := aEvMax[j];            // EVs: charge at maximum feasible power (EV energy
   has priority)
058   sumEv := sumEv + aEvReq[j];
059  END_FOR;
060  xCurtReq := FALSE;
061  rCurtReq := 0.0;
```

The selected EV policy is simple: charge as soon as possible at the maximum feasible power. It is not a complex deadline scheduler. Remaining time is still computed to prioritise shedding if the grid guard must reduce EV power.

Slack = steps until departure minus the steps needed at full charging power. Larger slack means a car can better tolerate being slowed down. A guard intervention can reduce EV charging to protect the grid; practice-data runs still delivered 100%.

6 · Strategy builds the planning question

FB_Strategy connects all the pieces. In normal operation, iStrategy = 1 selects forecasting and planning. Mode 0 is the simple baseline: charge the battery on solar surplus and discharge on deficit.



Day-ahead prices for tomorrow become visible after 13:00 (quarter-hour 52). Before that, the horizon ends at midnight; after that it may include tomorrow, but the configured 96-step computing limit still applies. The year-end horizon is shortened too.

aCi = energy price + import grid fee. aCe = max(0, 0.9 × energy price). Treating negative-price exports as having no benefit is paired with actual voluntary solar curtailment when exporting would cost money.

LEARNING GUIDE

7 · Forecasting: learn a normal day

The code contains no embedded future solar or load profiles. It learns from observations already available at the current step. It knows current load and available solar exactly; future values are estimates.

Memory	Purpose
aFec[day type, quarter-hour]	Expected fixed load for weekday / Saturday / Sunday
aSig[day type, quarter-hour]	Smoothed absolute load forecast error; a conservative reserve signal
aPv[quarter-hour]	Solar profile shared across day types
aVt[quarter-hour]	Most recently observed high/low tariff window
Recent actual/profile rings	Last four observations for intraday correction

Exponential moving average (EMA)

```
new profile = old profile + alpha × (observation − old profile)

load alpha = 0.5     solar alpha = 0.3
deviation alpha = 0.3
```

Actual ST • FB_Forecast.st • lines 95–104

```
095   IF aFecOk[dtp, q] THEN rPrevF := aFec[dtp, q]; ELSE rPrevF := rFec; END_IF;
096   IF aPvOk[q] THEN rPrevP := aPv[q]; ELSE rPrevP := rPv; END_IF;
097   IF aFecOk[dtp, q] THEN
098     aSig[dtp, q] := aSig[dtp, q] + Team_Cfg.FC_ALPHA_SIG * (ABS(rFec - aFec[dtp, q]) -
   aSig[dtp, q]);
099     aFec[dtp, q] := aFec[dtp, q] + Team_Cfg.FC_ALPHA_FEC * (rFec - aFec[dtp, q]);
100   ELSE
101     aFec[dtp, q] := rFec;
102     aFecOk[dtp, q] := TRUE;
103   END_IF;
104   IF aPvOk[q] THEN
```

The variable called sigma is a smoothed absolute deviation, not a statistically exact standard deviation. Cold start matters: when no matching slot has been learned, the load estimate falls back to a learned weekday slot or the highest load observed so far.

LEARNING GUIDE

8 · Forecasting: adjust for today

A learned profile alone would react slowly to a cloudy day or unusually busy site. Intraday correction compares recent actual observations to their pre-update profile values and scales the forecast accordingly.

Forecast calculation

```
load correction = clamp(0.5, recent actual / recent profile, 1.5)
solar correction = clamp(0.2, recent actual / recent profile, 1.5)
fade = exp(−future_step / 16)
future = profile × (1 + (correction − 1) × fade)

expected net load = fixed load + predicted EV power − solar
pessimistic net = expected + ksig × deviation + (1 − pvf) × solar
```

Setting	S1: two tariff	S2: day ahead
ksig (load reserve)	0.75	0
pvf (solar share trusted)	0.6	0.2
Peak penalty uses	Pessimistic net load	Expected net load

At future step zero, the forecast uses current actual load and solar, so expected and pessimistic net load are equal. Connected EVs are forecast as charging immediately; future arrivals are not magically known.

A forecast error is expected. That is why the controller replans every interval and why a separate grid guard checks the actual current interval.

LEARNING GUIDE

9 · Find a plausible monthly peak cap

A large monthly import spike creates a peak charge. If the month has already reached 600 kW, lowering today's import to 500 kW cannot undo that paid peak. The planner therefore starts with the month's peak so far as its cap floor.

F_PeakCap in pseudocode

```
test(cap):
  energy = current usable battery energy
  for each predicted interval:
    if net_load > cap:
      discharge enough to hold cap
      fail if power is too high or energy becomes negative
    else:
      recharge below cap, respecting power and capacity
  succeed

if test(already_paid_peak): return already_paid_peak
otherwise bisect between that peak and the import limit
return feasible cap × 1.02, clamped to the import limit
```

Bisection repeatedly halves a search range. The code tests the paid peak first, then performs up to 12 refinements. This is a short feasibility simulation, not the full dynamic program.

Actual ST • F_PeakCap.st • lines 28–35

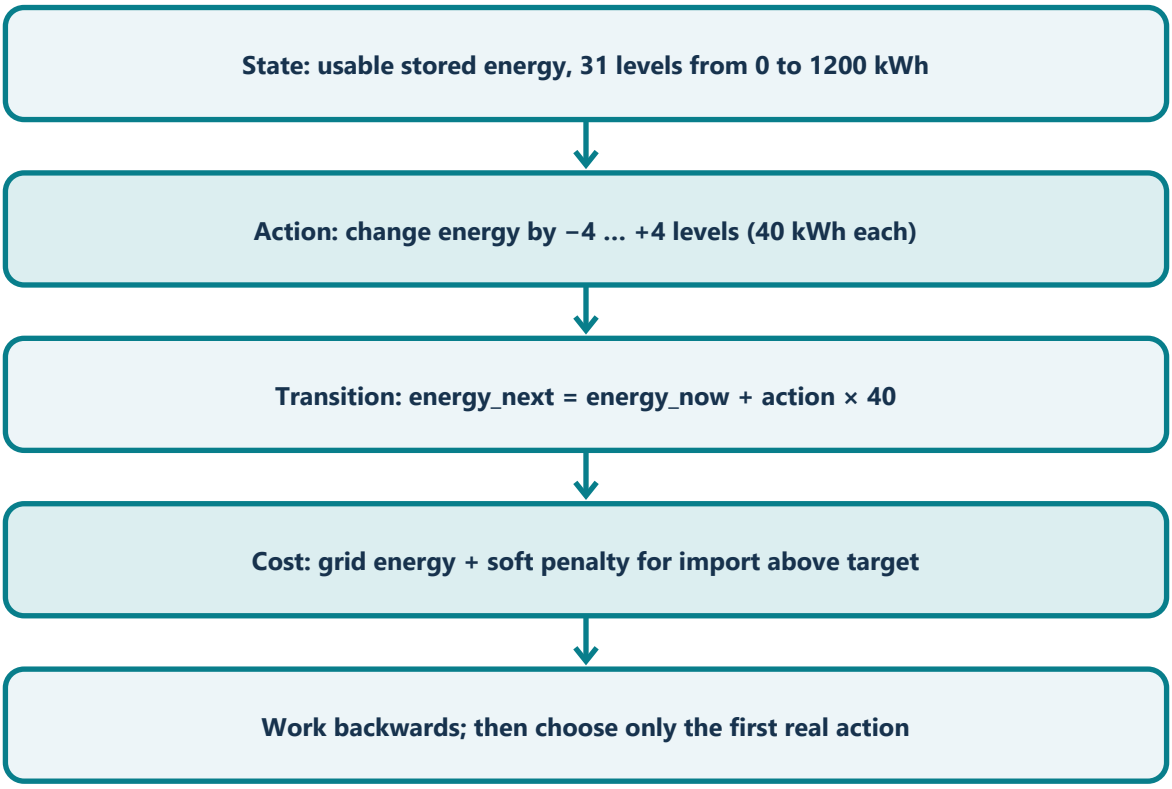
```
028  F_PeakCap := rFloor;
029  rLo := rFloor;
030  rHi := rLimImp;
031  FOR n := 0 TO 12 DO
032   IF n = 0 THEN
033     rMid := rFloor;
034   ELSE
035     rMid := 0.5 * (rLo + rHi);
```

This cap is a planning target, not the legal grid limit. A soft penalty discourages crossing it. The independent GridGuard enforces the physical grid constraints.

LEARNING GUIDE

10 · Dynamic programming: the map

The planner compares battery choices across a horizon, rather than choosing only what looks cheapest this minute. Its dynamic program stores the best future cost for each possible battery energy level.



A positive DP action increases stored energy and means charging; the final output rPb uses the opposite convention: positive means bus-side discharge. Efficiencies convert between stored-energy changes and bus power.

Model predictive control: calculate a whole future plan, apply just its first action, then recalculate using the next measurements. The future plan is guidance, not a fixed timetable.

LEARNING GUIDE

11 · Dynamic programming: the arithmetic

Backward recurrence

```
V[last horizon, energy] = −energy × terminal energy value

for time from horizon−1 backwards to 1:
  for each allowed action:
    calculate bus power and stage cost
  for each of 31 energy states:
    Vnow[state] = minimum over feasible actions of:
      stage_cost[action] + Vnext[state + action]
  Vnext = Vnow
```

Actual ST • FB_Planner.st • lines 102–118

```
102  FOR iE := 0 TO NS DO
103   V[iE] := -INT_TO_REAL(iE) * rDE * rTv;
104  END_FOR;
105  FOR a := AMIN TO AMAX DO
106   IF a > 0 THEN
107     aBus[a] := -INT_TO_REAL(a) * rDE / (ts * etaC);
108   ELSE
109     aBus[a] := -INT_TO_REAL(a) * rDE * etaD / ts;
110   END_IF;
111  END_FOR;
112  FOR i := h - 1 TO 1 BY -1 DO
113   rCiS := aCi[i] * (1.0 + rLateEps * INT_TO_REAL(i) / 96.0);   // tie-breaker scaled import
   price
114   FOR a := AMIN TO AMAX DO
115     IF NOT Team_Cfg.PLAN_TSPLIT THEN
116       IF xHiInDp THEN
117         rG := aNetHi[i] - aBus[a];
118       ELSE
```

The terminal value is 0.9 × the average import price over the horizon. Crediting leftover stored energy reduces the temptation to empty the battery just because the calculation stops at midnight.

Stage cost is import energy × import price, or minus export energy × export value, plus a strong penalty when planned import exceeds the chosen target. That penalty is an internal planning heuristic, not the organiser's actual peak invoice formula.

The S1 late-discharge tie-breaker increases future import prices by a small factor, 1 + 0.05 × i/96. Within otherwise similar tariff periods, this favours saving battery discharge for later. It is off in S2.

LEARNING GUIDE

12 · The first action is more precise

The future DP uses a coarse 40 kWh energy grid for speed. The action taken now is evaluated more precisely against current SoC and exact current load/solar.

First action in pseudocode

```
candidates = 41 powers from max_charge to max_discharge
also try:
  battery power that makes grid power exactly zero
  battery power that makes import exactly equal to cap

for each feasible candidate:
  calculate resulting stored energy
  interpolate the future DP cost between neighbouring levels
  add exact first-interval stage cost
choose the cheapest candidate
if import can be kept at cap: enforce that final correction
```

Actual ST • FB_Planner.st • lines 144–155

```
144  FOR m := 0 TO N_FIRST - 1 DO
145   aCand[nCand] := -rBcMax + (rBcMax + rBdMax) * INT_TO_REAL(m) / INT_TO_REAL(N_FIRST - 1);
146   nCand := nCand + 1;
147  END_FOR;
148  IF aNet[0] > -rBcMax AND aNet[0] < rBdMax THEN
149   aCand[nCand] := aNet[0];            // exact self-consumption (grid = 0)
150   nCand := nCand + 1;
151  END_IF;
152  IF aNet[0] - rT > -rBcMax AND aNet[0] - rT < rBdMax THEN
153   aCand[nCand] := aNet[0] - rT;       // import exactly at the cap
154   nCand := nCand + 1;
155  END_IF;
```

Interpolation avoids rounding today's energy to a single coarse level. There are 41 evenly spaced candidates and up to two extra exact-balance candidates. This explains why the candidate array has room for 43 entries.

The result is a request, not the final command. GridGuard still clamps it to physical limits and checks current import/export.

LEARNING GUIDE

13 · What v7 adds: a learned cap floor

V7 avoids repeatedly choosing an unrealistically low S2 import target early in a month. A slightly higher target can make cheap charging possible and reduce costly later peaks. This is deliberate trade-off, not a relaxation of the 1000 kW grid limit.

FB_CapFloor in pseudocode

```
R_day = lowest cap that could have served a measured day
        starting with a full battery

floor = min(import_limit,
            0.95 × mean(last up to 3 completed monthly peaks),
            1.20 × max(last up to 14 daily R_day values))

activate only in S2, with at least one completed month
and at least five daily R_day samples
planner target = max(original target, floor)
```

The code records current fixed load minus available solar, then adds the previous interval's actual EV draw when that measurement becomes available. It closes the day before starting the new day's record.

F_DayReq runs its 14-step bisection when quarter-hour ≥ 14 on the next day (03:30). The verified ST runs that calculation in one call; the delay does not mean one bisection iteration is spread across each of 14 PLC calls.

Actual ST • FB_CapFloor.st • lines 111–123

```
111  rFloor := 0.0;
112  IF xEnable AND nPk > 0 AND nRd >= 5 THEN
113   rSum := 0.0;
114   FOR i := 0 TO nPk - 1 DO
115     rSum := rSum + aPk[i];
116   END_FOR;
117   rMaxRd := 0.0;
118   n := MIN(nRd, 14);
119   FOR i := 1 TO n DO
120     rMaxRd := MAX(rMaxRd, aRd[(iRdHead - i + 14) MOD 14]);
121   END_FOR;
122   rFloor := MIN(rLimImp, MIN(Team_Cfg.CAPF_F * rSum / INT_TO_REAL(nPk), Team_Cfg.CAPF_C *
   rMaxRd));
123  END_IF;
```

In S1 the output floor is zero. History collection still occurs. The floor is based only on past realised measurements and completed monthly peaks.

LEARNING GUIDE

14 · GridGuard is the last safety net

Optimisation is allowed to be approximate. Current-interval grid protection cannot wait for last interval's grid meter: it predicts the bus balance from current load, solar, EV requests and feasible battery power.

GridGuard in pseudocode

```
clamp all requests to physically feasible values
predicted_grid = fixed_load − solar + EV_sum − battery

if predicted import is too high:
  restore voluntarily curtailed solar
  increase battery discharge / reduce charging
  reduce EV power, largest slack first
  flag failure if still above the margin target

if predicted export is too high:
  reduce discharge / increase charging
  curtail solar if needed
recalculate predicted grid power from final values
```

Configured margins are 1 kW: the target import limit is 999 kW and export target is 498 kW, below the hard 1000/499 kW trip limits. The margin is useful headroom, but the code can report that it cannot hold the margin when the underlying physical situation makes it impossible.

Actual ST • Team_Main.st • lines 148–155

```
148       stCmd.BESS_enable_discharge := TRUE;
149       stCmd.BESS_P_discharge := fbGuard.rPb;
150     ELSIF fbGuard.rPb < -Team_Cfg.P_EPS THEN
151       stCmd.BESS_enable_charge := TRUE;
152       stCmd.BESS_P_charge := -fbGuard.rPb;
153     END_IF;
154     FOR j := 0 TO 3 DO
155       stCmd.EV[j].P_charge := fbGuard.aEv[j];
```

The command starts from an all-zero structure. The IF / ELSIF means charge and discharge enables cannot both become true. Every command member is assigned each call through the held complete structure.

LEARNING GUIDE

15 · Walk through one interval

Known at interval k	Example value
Fixed consumption	1100 kW
Available solar	100 kW
EV requests	Two connected vehicles × 22 = 44 kW
Planner battery request	0 kW
Battery feasible discharge	At least 45 kW

This is an illustrative arithmetic case, not a recorded dataset interval. First predict grid import: 1100 − 100 + 44 − 0 = 1044 kW. That exceeds the guard's 999 kW target.

Worked balance

```
required correction = 1044 − 999 = 45 kW
final battery power = +45 kW (discharge)
final predicted import = 1100 − 100 + 44 − 45 = 999 kW

Over the 0.25 h interval:
  imported energy = 999 × 0.25 = 249.75 kWh
  battery bus delivery = 45 × 0.25 = 11.25 kWh
  stored energy reduction = 11.25 / 0.95 = 11.842 kWh
  EV energy into vehicles = 44 × 0.25 × 0.9 = 9.90 kWh
```

At k+1, log the actual flows of this interval. If actual import was 999 kW, the import energy logger adds 249.75 kWh. At an energy price of €0.10/kWh and high grid tariff €0.020811/kWh, this interval adds €24.975 purchase cost and about €5.198 grid cost.

If the month previously peaked at 980 kW, the new 999 kW peak increases the peak bill by (999 − 980) × €3.865 = €73.435. This is why one brief power spike matters.

LEARNING GUIDE

16 · Energy accounting is not control

FB_EnergyLogger sums what actually happened, not what the optimiser hoped would happen. It keeps a daily array of 365 rows, monthly array of 12 rows and one yearly aggregate.

Energy logging

```
energy increment [kWh] = actual power [kW] × 0.25 h

for the correct previous day and month:
  add PV, fixed load, charge, discharge, import and export
  add EV energy per charger and total
  peak = max(previous peak, actual import power)
copy yearly values to Team_Results
```

Actual ST • FB_EnergyLogger.st • lines 67–76

```
067   // EV per charger and total
068   FOR j := 0 TO 3 DO
069     eEv := REAL_TO_LREAL(aEv[j]) * ts;
070     Team_Log.aDay[d].E_ev_ch[j] := Team_Log.aDay[d].E_ev_ch[j] + eEv;
071     Team_Log.aDay[d].E_ev := Team_Log.aDay[d].E_ev + eEv;
072     Team_Log.aMonth[m].E_ev_ch[j] := Team_Log.aMonth[m].E_ev_ch[j] + eEv;
073     Team_Log.aMonth[m].E_ev := Team_Log.aMonth[m].E_ev + eEv;
074     Team_Log.stYear.E_ev_ch[j] := Team_Log.stYear.E_ev_ch[j] + eEv;
075     Team_Log.stYear.E_ev := Team_Log.stYear.E_ev + eEv;
076   END_FOR;
```

LREAL accumulators reduce round-off error over 35,040 additions. Calculations used for planning can be REAL for performance, while official totals need higher numerical care.

Team_Log is the detailed history. Team_Results is the official yearly interface with required names such as E_fne and E_import. The HMI can show copies of selected rows; those copies are not the authoritative energy history.

EV energy here is charger bus-side energy. The simulator separately tracks energy delivered into vehicles, including its EV efficiency. Comparing those numbers without understanding that distinction can look like a bug when it is not.

LEARNING GUIDE

17 · The actual electricity bill

Component	What the logger calculates
Purchase	Import kWh × energy price
Export revenue	Export kWh × 0.9 × energy price
Grid fee	Import kWh × tariff fee; no fee on export
Monthly peak	Maximum monthly import kW × €3.865/kW
Daily total	Purchase − revenue + grid fee (no peak allocation)
Monthly/yearly total	Purchase − revenue + grid fee + peak

Actual ST • FB_CostLogger.st • lines 39–41

```
039   cBuy := REAL_TO_LREAL(rImp) * ts * REAL_TO_LREAL(rPrice);
040   cRev := REAL_TO_LREAL(rExp) * ts * REAL_TO_LREAL(rPrice) *
   REAL_TO_LREAL(SimParam.EXPORT_FACTOR);
041   cGrid := REAL_TO_LREAL(rImp) * ts * fee;
```

High-tariff grid fee is €0.020811/kWh; low tariff is €0.010404/kWh. The energy price itself comes from the selected two-tariff or day-ahead table.

The cost logger must run after the energy logger because it reads the updated monthly import peak. Yearly peak cost is the sum of all 12 monthly peak costs, not the single highest annual peak times one price.

A negative price can make import purchase cost negative. Export revenue can also be negative, which adds cost after subtraction. The runtime logger uses the real price formula; the strategy tries to avoid unwanted export at negative prices.

LEARNING GUIDE

18 · Why remember previous values?

At the start of k, current fixed load belongs to k, but actual PV, battery and grid flows describe k−1. Pairing them directly would create an accounting error, particularly across midnight or month boundaries.

At k	Correct source when logging k−1
Fixed load	Remembered rFecPrev
Date / tariff	Remembered iDayPrev, iMonthPrev, xHighPrev
Actual PV / BESS / grid	Simulator's previous-step actual values
EV departure interval	Actual EV power, with the special correction below

Actual ST • Team_Main.st • lines 79–83

```
079     IF xLogNow AND aEvConnPrev[j] AND (k >= aEvDepKPrev[j]) THEN
080       aEvLast[j] := aEvCmdPrev[j];
081     ELSE
082       aEvLast[j] := simOut.EV[j].P_charge_act;
083     END_IF;
```

The simulator clears EV actual power when the vehicle departs, even though the just-completed interval still counted charging energy. The controller therefore substitutes its previous feasible, guard-approved EV command for that departure interval.

This is a narrow workaround for a simulator publication quirk. It does not generally replace actual measurements with requested values. The command used was already clamped to the same physical feasibility limits.

The final k=35,040 call is essential: it logs interval 35,039. Reset clears the histories before any old interval can leak into a new year.

LEARNING GUIDE

19 · HMI support: data before screens

FB_HmiUpdate publishes a convenient view of controller data into Team_HMI. These variables support native visualisation or WebVisu screens, but this frozen source snapshot does not contain a completed browser dashboard.

View	What it should explain to the operator
Header	Simulated date/time, progress, tariff and price mode
Power flows	Requests for k versus actual flows for k−1
Battery and EVs	SoC, direction, charger state, departure and remaining energy
Tables	Selected day/month and all days of selected month
Trends	96 samples = 24 simulated hours
Alarms	Current flags plus timestamped message history

Actual ST • FB_HmiUpdate.st • lines 194–200

```
194     Team_HMI.aTrFne[95] := stOut.FNE_P_act;
195     Team_HMI.aTrFec[95] := rFecPrev;
196     Team_HMI.aTrGrid[95] := stOut.EGC_P_import - stOut.EGC_P_export;
197     Team_HMI.aTrBess[95] := stOut.BESS_P_discharge_act - stOut.BESS_P_charge_act;
198     Team_HMI.aTrEv[95] := Team_HMI.rEvActSum;
199     Team_HMI.aTrSoc[95] := stOut.BESS_SoC;
200     Team_HMI.aTrPrice[95] := rPricePrev;
```

Trend arrays shift left once per new simulated step. Index 95 is the newest sample, index 0 the oldest. The SoC trend stores the current starting SoC, which is the previous interval's ending state; flow samples are actual interval k−1 values.

A WebVisu browser only displays values and sends allowed button writes. The battery optimiser still runs in the PLC/runtime. HMI writes to simulator controls must go through SimCtrl; selection variables choose which logged rows to display.

LEARNING GUIDE

20 · Alarms and diagnostic evidence

Alarms are not all failures. Some messages report normal protective actions, such as solar curtailment to avoid export overload. The code detects rising edges so a persistent condition does not fill history every interval.

Code	Meaning
1 / 2	Actual import / export overload
3 / 4	Simulation tripped / finished
5	EV left with undelivered energy
6	Grid protection reduced EV charging
7	Grid protection curtailed solar
8	Predicted import still exceeds the guard target

Actual ST • F_AlarmPush.st • lines 13–22

```
013  FOR i := 49 TO 1 BY -1 DO
014   Team_HMI.aAlarmHist[i] := Team_HMI.aAlarmHist[i - 1];
015  END_FOR;
016  Team_HMI.aAlarmHist[0].k := k;
017  Team_HMI.aAlarmHist[0].sStamp := sStamp;
018  Team_HMI.aAlarmHist[0].iCode := iCode;
019  Team_HMI.aAlarmHist[0].sText := sText;
020  Team_HMI.nAlarms := Team_HMI.nAlarms + 1;
021  Team_HMI.sLastAlarm := sText;
022  F_AlarmPush := TRUE;
```

The history holds 50 messages, newest at index 0. When a new message arrives, older entries shift toward index 49 and the oldest is discarded. nAlarms is a cumulative count, so it may exceed the number of retained entries.

Useful diagnostic counters include guard interventions, both battery enables, prediction error and step counts. Compare a stored prediction for k−1 with actual grid flow for k−1: matching timestamps matters as much as matching units.

LEARNING GUIDE

21 · What the verified results prove

Measure	S1 — two tariff	S2 — day ahead
Annual cost	€204,598.62	€169,833.86
Year completed	Yes	Yes
Grid trip	No	No
EV delivery	100.000%	100.000%
Max logged deviation	0.00001%	0.00000%
Change from v6	Unchanged	−€355.50 (0.209%)

These are practice-data simulator results, not a guarantee for every unseen dataset or real installation. The v7 candidate came from a robustness evaluation, and this port was then confirmed in Machine Expert.

The LP lower-bound reference uses perfect foresight: about €199,779 for S1 and €162,592 for S2. Our rolling controller does not call that LP or know future load/solar. Its gap is about 2.41% and 4.45%, respectively; part of the gap is the unavoidable value of better future knowledge.

Steady 5 ms stress-run throughput was roughly 5.8–6.0 ms per interval including simulator work. S1 took 216 s/year; S2 263 s/year with a slower start. These are wall-time throughput estimates, not instrumented worst-case PLC execution times.

Open delivery issues: complete/test/package the HMI; verify WebVisu acceptance with organisers; resolve evaluation price-table assumptions. Keep the verified archive intact while trying later optimisations.

22 · Every file has a job (1 of 2)

File	Responsibility
Team_Main.st	Sequences reset, previous-step logging, control, memory and HMI calls
FB_Strategy.st	Chooses horizon, EV requests, forecast, prices, cap floor and battery planner
FB_Forecast.st	Learns load/solar profiles and generates expected/pessimistic net load
FB_Planner.st	Calculates cap and backward DP, then chooses continuous first action
FB_CapFloor.st	Learns bounded S2 cap floor from daily requirements and completed peaks
FB_GridGuard.st	Clamps requests and corrects current import/export violations
FB_EnergyLogger.st	Accumulates actual daily/monthly/yearly energy and import peaks
FB_CostLogger.st	Accumulates real purchase, revenue, grid and peak costs
FB_HmiUpdate.st	Publishes screen values, selected rows, trends and alarm edges
F_PeakCap.st	Searches for lowest forecast-feasible cap from current battery energy
F_DayReq.st	Searches realised-day cap assuming initially full battery
F_StageCost.st	Planner’s energy cost plus soft cap penalty
F_EnergyPrice.st	Selects two-tariff or day-ahead price for a given interval

23 · Every file has a job (2 of 2)

File	Responsibility
F_BessMaxChg.st	Feasible charge limit from SoC, efficiency and remaining room
F_BessMaxDch.st	Feasible discharge limit from SoC, efficiency and usable energy
F_DayType.st	Maps weekday number to weekday/Saturday/Sunday class
F_DtString.st	Formats simulated date/time; does not read system clock
F_Pad2.st	Adds a leading zero for date/time components
F_AlarmPush.st	Inserts one message into the 50-entry history
Team_Cfg.st	Tuning constants and safety margins
Team_Prices.st	Two-tariff prices and 35,040 day-ahead prices; no load/PV profile
Team_HMI.st	Public display/selection/trend/alarm variables
Team_Log.st	Daily/monthly/yearly aggregate storage
Team_Results.st	Official yearly result interface plus equivalent cost names
typeTeamAgg.st	Data type for one energy/cost/peak aggregate
typeTeamAlarm.st	Data type for one timestamped alarm/message

Reading order: Team_Main → FB_Strategy → FB_Forecast → FB_Planner → FB_GridGuard. Then read loggers and HMI support. Use the following reference pages to recognise the smaller helpers and data declarations.

ACTUAL CODE EXCERPTS

Reference 1 · Battery feasibility helpers

LREAL intermediate arithmetic mirrors simulator clamps. The final function return is REAL, matching the strategy interface.

Actual ST • F_BessMaxChg.st • lines 7–9

```
007  F_BessMaxChg := LREAL_TO_REAL(MAX(0.0, MIN(REAL_TO_LREAL(SimParam.BESS_MAX_CHG),
008   (REAL_TO_LREAL(SimParam.BESS_SOC_MAX) - REAL_TO_LREAL(rSoc)) / 100.0 *
   REAL_TO_LREAL(SimParam.BESS_CAPACITY)
009   / (REAL_TO_LREAL(SimParam.TS_H) * REAL_TO_LREAL(SimParam.BESS_ETA_CHG)))));
```

Actual ST • F_BessMaxDch.st • lines 7–9

```
007  F_BessMaxDch := LREAL_TO_REAL(MAX(0.0, MIN(REAL_TO_LREAL(SimParam.BESS_MAX_DCH),
008   (REAL_TO_LREAL(rSoc) - REAL_TO_LREAL(SimParam.BESS_SOC_MIN)) / 100.0 *
   REAL_TO_LREAL(SimParam.BESS_CAPACITY)
009   * REAL_TO_LREAL(SimParam.BESS_ETA_DCH) / REAL_TO_LREAL(SimParam.TS_H))));
```

ACTUAL CODE EXCERPTS

Reference 2 · Prices and tariff selection

The enormous day-ahead initializer is not reproduced here. Team_Prices contains 35,040 REAL prices, not an embedded FEC or PV profile. In IEC initializers, repetition notation can compress repeated price values.

Actual ST • F_EnergyPrice.st • lines 14–23

```
014  IF iMode = 1 THEN
015   F_EnergyPrice := Team_Prices.aDA[LIMIT(0, k, 35039)];
016  ELSE
017   iQ := LIMIT(1, (iMonth - 1) / 3 + 1, 4);
018   IF xHigh THEN
019     F_EnergyPrice := Team_Prices.TT_VT[iQ];
020   ELSE
021     F_EnergyPrice := Team_Prices.TT_NT[iQ];
022   END_IF;
023  END_IF;
```

Actual ST • Team_Prices.st • lines 3–6

```
003  VAR_GLOBAL CONSTANT
004   // Elektra two-tariff energy price [EUR/kWh] per calendar quarter (1 = Jan-Mar ... 4 =
   Oct-Dec)
005   TT_NT : ARRAY[1..4] OF REAL := [0.09, 0.1051, 0.0934, 0.0949];
006   TT_VT : ARRAY[1..4] OF REAL := [0.1531, 0.1786, 0.1588, 0.1613];
```

ACTUAL CODE EXCERPTS

Reference 3 · Date and small formatting helpers

Dates come from the simulator's DT value. F_DtString converts that supplied timestamp to a human-readable date; F_Pad2 ensures two-digit fields. No wall clock is consulted.

Actual ST • F_DtString.st • lines 16–32

```
016  // ---IMPL---
017  udSec := DT_TO_UDINT(dtIn);                          // seconds since 1970-01-01
018  uH := UDINT_TO_USINT((udSec MOD 86400) / 3600);
019  uM := UDINT_TO_USINT((udSec MOD 3600) / 60);
020  iDoy := UDINT_TO_INT(udSec / 86400);
021  iYear := 1970;
022  xLeap := FALSE;
023  WHILE iYear < 2200 DO
024   xLeap := ((iYear MOD 4) = 0 AND (iYear MOD 100) <> 0) OR ((iYear MOD 400) = 0);
025   IF xLeap THEN iDim := 366; ELSE iDim := 365; END_IF;
026   IF iDoy < iDim THEN EXIT; END_IF;
027   iDoy := iDoy - iDim;
028   iYear := iYear + 1;
029  END_WHILE;
030  iMon := 1;
031  WHILE iMon < 12 DO
032   CASE iMon OF
```

Actual ST • F_Pad2.st • lines 6–11

```
006  // ---IMPL---
007  IF v < 10 THEN
008   F_Pad2 := CONCAT('0', USINT_TO_STRING(v));
009  ELSE
010   F_Pad2 := USINT_TO_STRING(v);
011  END_IF;
```

ACTUAL CODE EXCERPTS

Reference 4 · Aggregate and history data types

One aggregate row contains energies, four charger energies, import peak and cost components. One alarm record contains a simulated step, stamp, code and text.

Actual ST • typeTeamAgg.st • lines 4–20

```
004  STRUCT
005   E_fne : LREAL;                    // PV production after curtailment (FNE_P_act)
006   E_fec : LREAL;                    // fixed consumption (FEC_P_fix remembered from k-1)
007   E_bess_chg : LREAL;               // battery charging, bus side
008   E_bess_dch : LREAL;               // battery discharging, bus side
009   E_ev : LREAL;                     // all EV chargers, bus side
010   E_ev_ch : ARRAY[0..3] OF LREAL;   // per EV charger, bus side
011   E_import : LREAL;                 // grid import
012   E_export : LREAL;                 // grid export
013   P_peak : LREAL;                   // [kW] max 15-min import (monthly value; also kept
   per day for info)
014   C_purchase : LREAL;               // energy purchase = import * c_en
015   C_revenue : LREAL;                // export revenue = export * 0.9 * c_en
016   C_grid : LREAL;                   // grid fee on import (VT/NT)
017   C_peak : LREAL;                   // peak power cost (month/year only; 0 for days)
018   C_total : LREAL;                  // purchase - revenue + grid (+ peak for month/year)
019  END_STRUCT
020  END_TYPE
```

Actual ST • typeTeamAlarm.st • lines 3–9

```
003  STRUCT
004   k : DINT;                         // simulation step when raised
005   sStamp : STRING(20);              // simulated date/time "DD.MM.YYYY HH:MM"
006   iCode : INT;                      // see Team_HMI alarm code list
007   sText : STRING(80);               // human readable message
008  END_STRUCT
009  END_TYPE
```

ACTUAL CODE EXCERPTS

Reference 5 · Logs and official totals

Team_Log stores detailed history. Team_Results exposes the year totals that are compared to the organiser's SimScore totals.

Actual ST • Team_Log.st • lines 2–6

```
002  VAR_GLOBAL
003   aDay : ARRAY[0..364] OF typeTeamAgg;    // day_of_year 0..364 (C_peak = 0, C_total =
   purchase - revenue + grid)
004   aMonth : ARRAY[1..12] OF typeTeamAgg;   // month 1..12 incl. peak power P_peak and C_peak
005   stYear : typeTeamAgg;                   // yearly totals (copied to Team_Results)
006  END_VAR
```

Actual ST • Team_Results.st • lines 3–17

```
003  VAR_GLOBAL
004   E_fne : LREAL;                     // PV production after curtailment
005   E_fec : LREAL;                     // fixed consumption
006   E_bess_chg : LREAL;                // battery charge, bus side
007   E_bess_dch : LREAL;                // battery discharge, bus side
008   E_ev : LREAL;                      // EV chargers total, bus side
009   E_import : LREAL;                  // grid import
010   E_export : LREAL;                  // grid export
011   E_ev_ch : ARRAY[0..3] OF LREAL;    // per EV charger, bus side
012   P_peak : LREAL;                    // [kW] highest monthly peak of the year (info)
013   // cost components (spec 4.3)
014   C_purchase : LREAL;                // energy purchase
015   C_revenue : LREAL;                 // export revenue (0.9 * price)
016   C_grid : LREAL;                    // grid fee
017   C_peak : LREAL;                    // peak power cost = sum of 12 monthly values
```

ACTUAL CODE EXCERPTS

Reference 6 · HMI variables: display data

This is a data interface, not a screen definition. It makes the same underlying values easier to bind to gauges, text fields and trend objects. Read-only values must remain read-only from the HMI.

Actual ST • Team_HMI.st • lines 4–20

```
004  VAR_GLOBAL
005   // ---- status / header (show on every screen) ----
006   sDateTime : STRING(20);             // "DD.MM.YYYY HH:MM" start of current interval k
007   k : DINT;                           // current step
008   rProgressPct : REAL;                // 0..100 % of the year
009   xTariffHigh : BOOL;                 // TRUE = VT (high tariff) in current interval
010   sTariff : STRING(2);                // 'VT' / 'NT'
011   sPriceMode : STRING(20);            // 'Dvotarifni (HEP)' / 'Dan-unaprijed'
012   rPrice : REAL;                      // [EUR/kWh] energy price of the current interval
013   rGridFee : REAL;                    // [EUR/kWh] grid fee of the current interval
014   // ---- power flows: requested (simCmd, interval k) vs actual (*_act, interval k-1) [kW]
   ----
015   rFneAvail : REAL;                   // available PV in interval k (before curtailment)
016   rFneExpected : REAL;                // PV expected after our curtailment, interval k
017   xCurtActive : BOOL;                 // curtailment enabled in interval k
018   rCurtValue : REAL;                  // curtailment limit [kW]
019   rFneAct : REAL;                     // actual PV, interval k-1
020   rFec : REAL;                        // fixed consumption, interval k
```

Reference 7 · Tuning: forecast and basic safety

These constants tune expected behaviour. The legal physical plant parameters still come from SimParam. Changing a tuning value requires fresh simulator validation.

Actual ST • Team_Cfg.st • lines 3–20

```
003   N_STEPS : DINT := 35040;            // steps in the simulated year
004   GRID_MARGIN_IMP : REAL := 1.0;      // [kW] safety margin below EGC_MAX_IMPORT for the
   predicted grid power
005   GRID_MARGIN_EXP : REAL := 1.0;      // [kW] safety margin below EGC_MAX_EXPORT
006   P_EPS : REAL := 0.001;              // [kW] below this a BESS setpoint is treated as zero
007   // ---- forecast (FB_Forecast) ----
008   FC_ALPHA_FEC : REAL := 0.5;         // EMA weight of the newest day for the load profile
009   FC_ALPHA_PV : REAL := 0.3;          // EMA weight of the newest day for the PV profile
010   FC_ALPHA_SIG : REAL := 0.3;         // EMA weight for the FEC deviation (sigma)
011   FC_KSIG_TT : REAL := 0.75;          // two-tariff: pessimistic load = expected +
   FC_KSIG_TT * sigma
012   FC_KSIG_DA : REAL := 0.0;           // day-ahead: same (arbitrage is worth more than the
   reserve)
013   FC_CORR_W : INT := 4;               // intraday correction window [steps], 0 = off (max
   16)
014   FC_CORR_TAU : REAL := 16.0;         // fade-out of the intraday correction [steps]
015   FC_CORR_FEC : BOOL := TRUE;         // apply intraday correction to FEC
016   FC_CORR_PV : BOOL := TRUE;          // apply intraday correction to PV
017   // ---- planner (FB_Planner) ----
018   PLAN_H : INT := 96;                 // planning horizon [steps] (24 h)
019   PLAN_DA_PUBLISHED : BOOL := TRUE;   // TRUE: day-ahead horizon limited to published prices
    (13:00 rule)
020   PLAN_PEN : REAL := 20.0;            // [EUR/kW per step] penalty for planned import above
   the cap
```

ACTUAL CODE EXCERPTS

Reference 8 · Tuning: v7 and planner modes

The learned floor is enabled for day-ahead mode. PLAN_TSPLIT is FALSE in this version; its alternate branch remains in the code but is not the selected algorithm.

Actual ST • Team_Cfg.st • lines 24–34

```
024   CAPF_ON_DA : BOOL := TRUE;          // enable the floor in day-ahead mode (two-tariff:
   always off)
025   CAPF_F : REAL := 0.95;              // x mean of the last 3 paid monthly peaks
026   CAPF_C : REAL := 1.2;               // x max R_d of the last 14 days (bound)
027   CAPF_DELAY : INT := 14;             // R_d of yesterday usable from this qh of the new day
               // terminal value of stored energy = mean import price * PLAN_TVF
028   PLAN_PVF_TT : REAL := 0.6;          // two-tariff: PV share trusted in the pessimistic
   load (peak cap)
029   PLAN_PVF_DA : REAL := 0.2;          // day-ahead: lower -> higher cap -> more room for
   cheap grid charging
030   PLAN_HI_TT : BOOL := TRUE;          // two-tariff: DP peak penalty uses the pessimistic
   net load
031   PLAN_HI_DA : BOOL := FALSE;         // day-ahead: DP peak penalty uses the expected net
   load
032   PLAN_TSPLIT : BOOL := FALSE;        // TRUE: free import level from the expected forecast
   (see FB_Planner)
033   PLAN_CAPM : REAL := 1.02;           // safety factor on the computed peak cap
034  END_VAR
```

ACTUAL CODE EXCERPTS

Reference 9 · Planner stage cost

rG is expected net grid power. rGpen may use a more conservative forecast for penalty evaluation. The soft penalty is an optimisation aid, while the actual bill is calculated independently by FB_CostLogger.

Actual ST • F_StageCost.st • lines 15–22

```
015  IF rG >= 0.0 THEN
016   F_StageCost := rG * SimParam.TS_H * rCi;
017  ELSE
018   F_StageCost := -MIN(-rG, rLimExp) * SimParam.TS_H * rCe;
019  END_IF;
020  IF rGpen > rT THEN
021   F_StageCost := F_StageCost + Team_Cfg.PLAN_PEN * (rGpen - rT);
022  END_IF;
```

ACTUAL CODE EXCERPTS

Reference 10 · Realised-day cap feasibility

Every trial starts at full usable energy and walks the recorded day. This is a retrospective feasibility score; it does not claim that the real battery was full at midnight.

Actual ST • F_DayReq.st • lines 26–47

```
026  FOR n := 1 TO 14 DO
027   rM := 0.5 * (rA + rB);
028   xOk := TRUE;
029   rE := rEuse;                          // full battery at 00:00
030   FOR i := 0 TO 95 DO
031     rOver := aDay[i] - rM;
032     IF rOver > 0.0 THEN
033       rE := rE - rOver * SimParam.TS_H / SimParam.BESS_ETA_DCH;
034       IF rOver > SimParam.BESS_MAX_DCH OR rE < 0.0 THEN
035         xOk := FALSE;
036         EXIT;
037       END_IF;
038     ELSE
039       rE := rE + MIN(MIN(-rOver, SimParam.BESS_MAX_CHG), (rEuse - rE) / (SimParam.TS_H *
   SimParam.BESS_ETA_CHG))
040         * SimParam.TS_H * SimParam.BESS_ETA_CHG;
041     END_IF;
042   END_FOR;
043   IF xOk THEN
044     rB := rM;
045   ELSE
046     rA := rM;
047   END_IF;
```

Try explaining it back to yourself

If you can answer these questions, you understand the design rather than just recognising its block names.

- Why does the code log interval k−1 before choosing the action for k?
- Why does a positive battery setpoint reduce grid import?
- Why can 11.25 kWh delivered by the battery reduce stored energy by more than 11.25 kWh?
- Why does the optimiser remember the monthly peak already reached?
- What is the difference between a 999 kW guard target and a 1000 kW legal trip limit?
- How can the forecast learn without knowing tomorrow's solar profile?
- Why is the planner's terminal value useful?
- Why does a browser dashboard not replace the PLC program?
- What does the special EV-departure correction fix?
- Why must an efficiency optimisation be regression-tested even if it looks harmless?

A short answer to the whole system

"We learn a forecast from measurements, plan battery use across the available price horizon, protect the current grid balance, and account for actual energy and costs on simulated time. We repeat this every 15-minute interval."

Generated from the frozen v7 files. Supporting generator, editable Mermaid flowcharts and validation artifacts are in code-review/learning-pdf/. The source archive remains the authoritative executable deliverable.